WhatsApp Booking Bot - Complete System Documentation

Table of Contents

1. [System Overview](#system-overview)
2. [Architecture](#architecture)
3. [Database Schema](#database-schema)
4. [Core Functionalities](#core-functionalities)
5. [API Endpoints](#api-endpoints)
6. [WhatsApp Webhook Flow](#whatsapp-webhook-flow)
7. [Cron Jobs](#cron-jobs)
8. [Admin Dashboard](#admin-dashboard)
9. [Data Flow Diagrams](#data-flow-diagrams)
10. [Environment Variables](#environment-variables)
11. [Key Libraries & Dependencies](#key-libraries--dependencies)

System Overview

WhatsApp Booking Bot is a comprehensive community management system built with Next.js that enables residents to:
- Book community halls via WhatsApp
- Register and track complaints
- Check booking/complaint status
- View maintenance payment status
- Cancel bookings and complaints

The system includes an admin dashboard for managing residents, bookings, complaints, and system settings.

Tech Stack:
- Frontend: Next.js 14, React 18, TypeScript, Tailwind CSS, shadcn/ui
- Backend: Next.js API Routes, Server Actions
- Database: Supabase (PostgreSQL)
- Messaging: Twilio WhatsApp API
- PDF Generation: jsPDF with autoTable
- State Management: React Hooks, SWR for data fetching

Architecture

High-Level System Architecture

\`\`\`
┌──┐
│   WhatsApp Users   │

```
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                                │
                                                │
                                                │
```

```
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│                                Supabase (PostgreSQL)                                                         │
│                                                                                                              │
│  ┌──────────────────────────────────────────────────────────────────────────────────────────────────────┐   │
│  │  Tables:                                                                                             │   │
│  │  - profiles (Residents)                                                                              │   │
│  │  - bookings (Hall bookings)                                                                          │   │
│  │  - complaints (Resident complaints)                                                                  │   │
│  │  - booking_settings (System configuration)                                                           │   │
│  │  - maintenance_payments (Payment tracking)                                                           │   │
│                                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```
\`\`\`

Database Schema

1. **profiles** Table
Stores resident information and maintenance status.

\`\`\`sql
```sql
CREATE TABLE profiles (
  id UUID PRIMARY KEY,
  phone_number TEXT UNIQUE NOT NULL,
  name TEXT NOT NULL,
  cnic TEXT,
  apartment_number TEXT NOT NULL,
  building_block TEXT,
  is_active BOOLEAN DEFAULT true,
  maintenance_charges DECIMAL(10,2) DEFAULT 0,
  maintenance_paid BOOLEAN DEFAULT false,
  last_payment_date DATE,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```
\`\`\`

Key Fields:

- `phone_number`: WhatsApp phone number (unique identifier)
- `is_active`: Controls access to the system
- `maintenance_paid`: Current maintenance payment status
- `maintenance_charges`: Monthly maintenance amount

Indexes: phone_number, apartment_number, maintenance_paid

---

2. **bookings** Table
Stores community hall booking information.

```sql
CREATE TABLE bookings (
  id UUID PRIMARY KEY,
  profile_id UUID NOT NULL REFERENCES profiles(id),
  booking_date DATE NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  status TEXT DEFAULT 'confirmed'
    CHECK (status IN ('confirmed', 'cancelled', 'payment_pending')),
  booking_charges DECIMAL(10,2) DEFAULT 500,
  payment_status TEXT DEFAULT 'pending'
    CHECK (payment_status IN ('pending', 'paid')),
  payment_due_date DATE,
  reminder_last_sent_at TIMESTAMP,
  payment_confirmation_sent BOOLEAN DEFAULT false,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  UNIQUE(booking_date, start_time)
);
```

Key Fields:
- `status`: Booking confirmation status
- `payment_status`: Payment tracking (pending/paid)
- `reminder_last_sent_at`: Tracks when reminders were sent
- `payment_confirmation_sent`: Tracks if payment confirmation was sent

Indexes: profile_id, booking_date, payment_status

---

3. **complaints** Table
Stores resident complaints with grouping capability.

```sql
CREATE TABLE complaints (
```

```
  id UUID PRIMARY KEY,
  complaint_id TEXT UNIQUE NOT NULL,
  profile_id UUID NOT NULL REFERENCES profiles(id),
  category TEXT NOT NULL
    CHECK (category IN ('apartment', 'building')),
  subcategory TEXT NOT NULL,
  description TEXT,
  status TEXT DEFAULT 'pending'
    CHECK (status IN ('pending', 'in-progress', 'completed', 'cancelled')),
  group_key TEXT,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Key Fields:
- `complaint_id`: Auto-generated complaint identifier (CMP + timestamp)
- `category`: Apartment or Building level
- `subcategory`: Type of complaint (plumbing, electric, civil, other)
- `group_key`: For grouping similar complaints together
- `status`: Workflow status

Indexes: profile_id, status, group_key

---

4. **booking_settings** Table
Stores system configuration for bookings.

```sql
CREATE TABLE booking_settings (
  id UUID PRIMARY KEY,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  slot_duration_minutes INTEGER NOT NULL,
  working_days INTEGER[] NOT NULL,
  booking_charges INTEGER NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Key Fields:
- `start_time`: Hall opening time (e.g., 09:00)
- `end_time`: Hall closing time (e.g., 18:00)
- `slot_duration_minutes`: Duration of each booking slot (15, 30, 60, 120)
- `working_days`: Array of working days (1=Monday, 7=Sunday)
- `booking_charges`: Cost per booking

---

5. **maintenance_payments** Table
Tracks monthly maintenance payments.

```sql
CREATE TABLE maintenance_payments (
  id UUID PRIMARY KEY,
  profile_id UUID NOT NULL REFERENCES profiles(id),
  year INTEGER NOT NULL,
  month INTEGER NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  status TEXT DEFAULT 'unpaid'
    CHECK (status IN ('paid', 'unpaid')),
  paid_date DATE,
  reminder_last_sent_at TIMESTAMP,
  confirmation_sent BOOLEAN DEFAULT false,
  confirmation_sent_at TIMESTAMP,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Key Fields:
- `year`, `month`: Billing period
- `status`: Payment status
- `reminder_last_sent_at`: Tracks reminder sending

Core Functionalities

1. **WhatsApp Messaging System**

File: `lib/twilio.ts`

```typescript
sendWhatsAppMessage(to: string, body: string)
```

Purpose: Sends WhatsApp messages via Twilio API

Flow:
1. Validates Twilio credentials (TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE_NUMBER)
2. Formats phone number with "whatsapp:" prefix
3. Makes HTTP POST request to Twilio API

4. Returns success/error response

Error Handling: Logs errors and returns no-op response if credentials missing (for local dev)

---

2. **User State Management**

File: `app/api/webhook/route.tsx`

State Structure:
\`\`\`typescript
```
userStates = Map<phoneNumber, {
  step: string,           // Current conversation step
  type?: "booking" | "complaint" | "cancel" | "status",
  date?: string,          // Selected date
  slots?: TimeSlot[],     // Available time slots
  complaint?: {           // Complaint data
    category?: string,
    subcategory?: string,
    description?: string
  },
  cancelItems?: any[],    // Items to cancel
  statusItems?: any[]     // Items to check status
}>
```
\`\`\`

Purpose: Maintains conversation state for each user across multiple messages

---

3. **Booking Flow**

Steps:
1. User selects "Book Community Hall" (option 2)
2. User enters date in DD-MM-YYYY format
3. System validates:
   - Date is not in the past
   - Date is a working day
   - Slots are available
4. System displays available time slots
5. User selects a slot
6. System creates booking with:
   - Status: "confirmed"
   - Payment Status: "pending"
   - Booking Charges: 500 (default)
7. System sends invoice link via WhatsApp

8. Cron job sends reminders on Day 1 and Day 2
9. Cron job cancels booking on Day 3 if unpaid

Key Functions:
- `handleBookingFlow()`: Main booking flow handler
- `handleDateInput()`: Validates and processes date input
- `handleSlotSelection()`: Processes slot selection
- `generateTimeSlots()`: Creates available slots based on settings

---

4. **Complaint Flow**

Steps:
1. User selects "Register Complaint" (option 1)
2. User selects category (Apartment/Building)
3. User selects subcategory (Plumbing/Electric/Civil/Other)
4. If "Other", user provides description
5. System creates complaint with:
   - Auto-generated complaint_id (CMP + timestamp)
   - Status: "pending"
   - Group key for similar complaints
6. System confirms complaint creation

Key Functions:
- `handleComplaintFlow()`: Main complaint flow handler
- `createComplaint()`: Creates complaint in database
- `generateComplaintId()`: Generates unique complaint ID

---

5. **Status Check Flow**

Steps:
1. User selects "Check Status" (option 3)
2. System fetches:
   - Recent bookings (confirmed/payment_pending)
   - Recent complaints (not cancelled)
3. System displays list with status indicators
4. User selects item to view details

Key Functions:
- `initializeStatusCheck()`: Fetches user's items
- `handleStatusFlow()`: Processes status selection
- `displayStatusOptions()`: Formats status display

---

6. **Cancellation Flow**

Steps:
1. User selects "Cancel Booking/Complaint" (option 4)
2. System fetches:
   - Active bookings (future dates)
   - Active complaints (pending/in-progress)
3. User selects item to cancel
4. System asks for confirmation
5. System updates status to "cancelled"

Key Functions:
- `initializeCancelFlow()`: Fetches cancellable items
- `handleCancelFlow()`: Processes cancellation
- `processCancellation()`: Updates database

---

7. **Invoice Generation**

File: `lib/invoice.ts`

Functions:
- `generateBookingInvoicePdf()`: Creates booking invoice PDF
- `generateMaintenanceInvoicePdf()`: Creates maintenance invoice PDF

Invoice Components:
- Brand header with "Green Three" branding
- Invoice number (BK-YYYYMMDD-SUFFIX or MT-YYYYMM-SUFFIX)
- Resident details
- Payment details table
- Status stamp (PAID/UNPAID)
- Footer with generation date

PDF Libraries:
- jsPDF: PDF generation
- jspdf-autotable: Table formatting

API Endpoints

1. **POST /api/webhook**
Purpose: Handles incoming WhatsApp messages

Request:
- Form data from Twilio
- `From`: Sender's WhatsApp number

- `Body`: Message content
- `ProfileName`: Sender's name

Response:
- TwiML XML response with bot's reply

Flow:
1. Extract phone number and message
2. Fetch user profile from database
3. Check if user is active
4. Process message based on current state
5. Return TwiML response

Error Handling:
- Returns error message if user not registered
- Returns error message if user inactive
- Catches and logs all exceptions

---

2. **POST /api/bookings/update-payment-status**
Purpose: Updates booking payment status (Admin only)

Request Body:
\`\`\`json
{
  "bookingId": "uuid",
  "paymentStatus": "paid" | "pending"
}
\`\`\`

Response:
- Success: Updates booking and sends confirmation to resident
- Error: Returns error message

Side Effects:
- Sends WhatsApp message with invoice link
- Updates `payment_confirmation_sent` flag

---

3. **POST /api/complaints/update-status**
Purpose: Updates complaint status (Admin only)

Request Body:
\`\`\`json
{
  "complaintId": "uuid",

  "status": "pending" | "in-progress" | "completed" | "cancelled"
}
\`\`\`

Response:
- Success: Updates complaint and notifies resident
- Error: Returns error message

Side Effects:
- Sends WhatsApp notification to resident

---

4. **POST /api/cron/booking-reminder**
Purpose: Sends booking payment reminders (Cron job)

Trigger: Scheduled via external cron service

Logic:
- Fetches all confirmed bookings with pending payment
- For each booking:
  - Day 0: No action
  - Day 1-2: Send reminder (once per day)
  - Day 3+: Cancel booking and notify resident

Side Effects:
- Sends WhatsApp reminders
- Updates `reminder_last_sent_at`
- Cancels overdue bookings

---

5. **POST /api/cron/booking-confirmation**
Purpose: Sends payment confirmation messages (Cron job)

Logic:
- Fetches all paid bookings without confirmation sent
- Sends WhatsApp message with invoice link
- Updates `payment_confirmation_sent` flag

---

6. **POST /api/cron/maintenance-reminder**
Purpose: Sends maintenance payment reminders (Cron job)

Logic:
- Fetches unpaid maintenance payments
- Sends WhatsApp reminders

- Updates `reminder_last_sent_at`

---

7. **POST /api/cron/maintenance-confirmation**
Purpose: Sends maintenance payment confirmations (Cron job)

Logic:
- Fetches paid maintenance payments without confirmation
- Sends WhatsApp confirmation
- Updates `confirmation_sent` flag

WhatsApp Webhook Flow

Message Processing Flow

```
┌──────────────
```

```
│  - Status check flow          │
│  - Cancellation flow          │
└──────────────────────────────┘
          │
          ▼
┌──────────────────────────────┐
│  Generate response message   │
└──────────────────────────────┘
          │
          ▼
┌──────────────────────────────┐
│  Return TwiML XML response   │
└──────────────────────────────┘
```
\`\`\`

State Transitions

\`\`\`
Initial State
    │
    ├──→ "1" → complaint_category → complaint_subcategory → complaint_description → Create Complaint
    │
    ├──→ "2" → booking_date → waiting_for_slot → Create Booking
    │
    ├──→ "3" → status_list → Display Details
    │
    ├──→ "4" → cancel_list → cancel_confirmation → Process Cancellation
    │
    ├──→ "5" → Display Profile Info
    │
    └──→ "6" → Display Maintenance Status
\`\`\`

Cron Jobs

Cron Job Architecture

All cron jobs require `x-cron-key` header matching `CRON_SECRET` environment variable.

Cron Jobs:

Job	Route	Frequency	Purpose
Booking Reminder	`/api/cron/booking-reminder`	Every 6 hours	Send payment reminders, cancel overdue
Booking Confirmation	`/api/cron/booking-confirmation`	Every 12 hours	Send payment confirmations
Maintenance Reminder	`/api/cron/maintenance-reminder`	Daily	Send maintenance payment reminders
Maintenance Confirmation	`/api/cron/maintenance-confirmation`	Daily	Send maintenance confirmations

Booking Reminder Logic

```
For each confirmed booking with pending payment:

  Days since creation = (now - created_at) / 86400

  if days == 0:
    → Skip (same day as creation)

  if days >= 3:
    → Cancel booking
    → Send cancellation message

  if days in [1, 2]:
    → Check if reminder already sent today
    → If not sent today:
      → Send reminder message
      → Update reminder_last_sent_at
```

Admin Dashboard

File: `app/admin/page.tsx`

Dashboard Features

1. **Residents Management**
- View all residents with search and filter
- Add new resident
- Edit resident details

- Delete resident
- Toggle maintenance payment status
- View resident profile details
- Export residents report to PDF

Key Data:
- Name, Phone, Apartment, Maintenance Charges, Payment Status

---

2. **Bookings Management**
- View all bookings with filters
- Filter by date, status, payment status
- Update payment status
- Cancel booking
- View booking details
- Export bookings report to PDF

Key Data:
- Customer, Apartment, Date, Time, Amount, Payment Status

---

3. **Complaints Management**
- View individual complaints
- Filter by status, date
- Update complaint status
- View complaint details
- Export complaints report to PDF

Key Data:
- Complaint ID, Customer, Category, Type, Status, Created Date

---

4. **Complaint Analytics**
- View grouped complaints
- Identify common issues
- See affected residents
- Track complaint patterns

Grouping: By category + subcategory

---

5. **System Settings**
- Configure booking start/end time
- Set slot duration (15, 30, 60, 120 minutes)

- Set booking charges
- Select working days
- Save settings to database

---

6. **Quick Actions**
- Trigger booking reminders manually
- Trigger maintenance reminders manually
- Refresh all data
- Export reports (PDF)

Admin Dashboard State Management

```typescript
// Main state
const [settings, setSettings] = useState<BookingSettings>()
const [bookings, setBookings] = useState<Booking[]>()
const [complaints, setComplaints] = useState<Complaint[]>()
const [profiles, setProfiles] = useState<Profile[]>()
const [groupedComplaints, setGroupedComplaints] = useState<GroupedComplaint[]>()

// UI state
const [loading, setLoading] = useState(true)
const [saving, setSaving] = useState(false)
const [refreshing, setRefreshing] = useState(false)
const [searchTerm, setSearchTerm] = useState("")
const [dateFilter, setDateFilter] = useState("")
const [statusFilter, setStatusFilter] = useState("all")

// Modal state
const [isAddUserOpen, setIsAddUserOpen] = useState(false)
const [isEditUserOpen, setIsEditUserOpen] = useState(false)
const [editingUser, setEditingUser] = useState<Profile | null>()
```

Data Flow Diagrams

Booking Creation Flow

```
User sends date
    │
    ▼
```

```
Validate date format (DD-MM-YYYY)
    │
    ├── Invalid → Send error message
    │
    ▼
Check if date is in past
    │
    ├── Yes → Send error message
    │
    ▼
Fetch booking settings
    │
    ▼
Check if date is working day
    │
    ├── No → Send error message
    │
    ▼
Fetch existing bookings for date
    │
    ▼
Generate available time slots
    │
    ├── No slots → Send error message
    │
    ▼
Display slots to user
    │
    ▼
User selects slot
    │
    ▼
Double-check slot availability
    │
    ├── Taken → Send error message
    │
    ▼
Create booking in database
    │
    ▼
Send confirmation with invoice link
```

Payment Reminder Flow

```
Cron job triggered
    │
    ▼
Fetch all confirmed bookings with pending payment
    │
    ▼
For each booking:
    │
    ├─ Calculate days since creation
    │
    ├─ If day 0: Skip
    │
    ├─ If day 1-2:
    │   ├─ Check if reminder sent today
    │   ├─ If not: Send reminder
    │   └─ Update reminder_last_sent_at
    │
    └─ If day 3+:
        ├─ Cancel booking
        ├─ Send cancellation message
        └─ Update status to "cancelled"
```

Complaint Creation Flow

```
User selects complaint category
    │
    ▼
User selects subcategory
    │
    ├─ If "Other": Ask for description
    │   │
    │   ▼
    │   User provides description
    │
    ▼
Generate complaint ID (CMP + timestamp)
    │
    ▼
Generate group key (category_subcategory)
    │
    ▼
Create complaint in database
    │
    ▼
Send confirmation with complaint ID
```

\`\`\`

Environment Variables

Required Variables

\`\`\`env
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key

# Twilio
TWILIO_ACCOUNT_SID=your-account-sid
TWILIO_AUTH_TOKEN=your-auth-token
TWILIO_PHONE_NUMBER=whatsapp:+14155238886

# Cron Security
CRON_SECRET=your-secret-key

# App URL
NEXT_PUBLIC_APP_URL=https://your-app-url.com
\`\`\`

Optional Variables

\`\`\`env
# Development
NEXT_PUBLIC_DEV_SUPABASE_REDIRECT_URL=http://localhost:3000
\`\`\`

Key Libraries & Dependencies

Frontend
- **next**: 14.0.0 - React framework
- **react**: 18 - UI library
- **react-hook-form**: 7.54.1 - Form management
- **zod**: 3.24.1 - Schema validation
- **tailwindcss**: 3 - Styling
- **shadcn/ui**: Component library
- **lucide-react**: Icons
- **recharts**: 2.15.0 - Charts
- **date-fns**: 4.1.0 - Date utilities

Backend

- **@supabase/supabase-js**: 2.39.0 - Database client
- **twilio**: 4.19.0 - WhatsApp messaging
- **jspdf**: 3.0.3 - PDF generation
- **jspdf-autotable**: 5.0.2 - PDF tables

Development
- **typescript**: 5 - Type safety
- **eslint**: 8 - Code linting
- **postcss**: 8 - CSS processing

Key Functions Reference

Webhook Handler (`app/api/webhook/route.tsx`)

Function	Purpose
`getProfile()`	Fetch user profile by phone number
`processMessage()`	Route message to appropriate handler
`handleMainMenu()`	Process main menu selection
`handleBookingFlow()`	Handle booking flow
`handleComplaintFlow()`	Handle complaint flow
`handleStatusFlow()`	Handle status check flow
`handleCancelFlow()`	Handle cancellation flow
`generateTimeSlots()`	Generate available booking slots
`formatDate()`	Format date for display
`formatTime()`	Format time for display

Invoice Generation (`lib/invoice.ts`)

Function	Purpose
`generateBookingInvoicePdf()`	Generate booking invoice PDF
`generateMaintenanceInvoicePdf()`	Generate maintenance invoice PDF
`getBookingInvoiceNumber()`	Generate booking invoice number
`getMaintenanceInvoiceNumber()`	Generate maintenance invoice number
`formatCurrency()`	Format amount as currency

Admin Dashboard (`app/admin/page.tsx`)

Function	Purpose
`fetchProfiles()`	Fetch all residents
`fetchBookings()`	Fetch all bookings
`fetchComplaints()`	Fetch all complaints
`addNewUser()`	Add new resident
`editUser()`	Update resident details

`deleteUser()`	Delete resident
`updateMaintenanceStatus()`	Toggle maintenance payment
`updateComplaintStatus()`	Update complaint status
`updateBookingPaymentStatus()`	Update booking payment
`saveSettings()`	Save booking settings
`triggerBookingReminder()`	Manually trigger booking reminders
`triggerMaintenanceReminder()`	Manually trigger maintenance reminders

Error Handling

Webhook Error Handling
- User not registered: Return error message
- User inactive: Return error message
- Invalid input: Return error message with guidance
- Database errors: Log and return generic error message

API Error Handling
- Missing CRON_SECRET: Return 401 Unauthorized
- Database errors: Return 500 with error message
- Twilio errors: Log and continue processing

Admin Dashboard Error Handling
- Failed operations: Show toast notification
- Validation errors: Show validation messages
- Network errors: Show error toast

Security Considerations

1. **Row Level Security (RLS):** All tables have RLS enabled with permissive policies
2. **Cron Security:** All cron endpoints require `CRON_SECRET` header
3. **Phone Number Validation:** Phone numbers are unique and validated
4. **User Status:** Only active users can access the system
5. **Data Isolation:** Users can only see their own data via WhatsApp

---

Deployment Checklist

- [ ] Set all environment variables in Vercel
- [ ] Configure Twilio webhook URL to point to `/api/webhook`
- [ ] Set up external cron service to trigger cron endpoints
- [ ] Run database migration scripts
- [ ] Test WhatsApp messaging
- [ ] Test booking flow end-to-end

- [ ] Test admin dashboard
- [ ] Configure backup strategy for Supabase
- [ ] Set up monitoring and logging
- [ ] Test invoice generation and download

Troubleshooting

WhatsApp Messages Not Sending
- Check Twilio credentials in environment variables
- Verify phone number format (should include country code)
- Check Twilio account balance
- Review Twilio logs for errors

Bookings Not Creating
- Verify database connection
- Check if date is a working day
- Ensure slots are available
- Check for unique constraint violations

Cron Jobs Not Running
- Verify CRON_SECRET is set correctly
- Check external cron service configuration
- Review cron job logs
- Verify endpoint URLs are correct

Invoice Download Not Working
- Check if jsPDF library is loaded
- Verify booking/maintenance data exists
- Check browser console for errors
- Ensure PDF generation completes

Future Enhancements

1. **Multi-language Support:** Add support for Urdu and other languages
2. **Payment Gateway Integration:** Integrate Stripe/JazzCash for online payments
3. **Advanced Analytics:** Add more detailed reporting and analytics
4. **Mobile App:** Create native mobile app for better UX
5. **Notification Preferences:** Allow users to customize notification frequency
6. **Maintenance Scheduling:** Add maintenance request scheduling
7. **Document Management:** Store and manage resident documents
8. **Audit Logging:** Track all admin actions for compliance

Support & Contact

For issues or questions:
1. Check the troubleshooting section
2. Review error logs in Vercel
3. Check Supabase dashboard for database issues
4. Review Twilio logs for messaging issues
5. Contact development team for further assistance

\`\`\`

This comprehensive documentation covers all technical aspects of the WhatsApp booking bot system. It includes system architecture, database schema, all functionalities, API endpoints, data flows, and deployment information that will help new developers understand and work with the system effectively.